OneEvery

Conceptualized, Created and Developed by Mashy. All Rights Reserved.

Table of contents

Introduction 3
Welcome 4
What's new 5
Getting Started 5
TreeView 6
CLI 7
App 9
New App 10
Open App 11
Publish App 12
CSS Files 13
JavaScript Files 14
Pages 16
Creator 17
Frameworks 18
Devices 18
Categories 19
Styles 19
Tags 20
PlayGround 21
Templates 22
Attributes 22
Elements 23
Element 25
Attributes 27
Styles 29
CSS 30

Introduction

Introduction

OneEvery is a B4J application built with the idea to build websites initially using the Twitter BootStrap framework. The current framework that has been 'created' internal with OneEvery is TB4 (Twitter Bootstrap 4).

The concept of OneEvery is flexibility, meaning that any JavaScript framework / library should be easily imported into OneEvery and one can then

1. **Build** the various elements using html tags
2. **Create** templates to build pages (i.e. a collection of html elements with attributes and style (css) classes and finaly.
3. **Publish** (i.e create pages and then publish these as a website)

One has to be able to also create a project website, link css and javascript files, create pages, elements easily.

The most important feature of OneEvery is that **there should not be any hard coded code into it to build any html element and that building html elements should be as easy as possible.** This idea then means that OneEvery should not be locked to a single JavaScript framework.

Sequence of using OneEvery

Whilst OneEvery will come with BootStrap 4 built into it, one will be able to add any other framework to it. The sequence of using OneEvery initially is

1. Create a website project
2. Link CSS and JavaScript Files
3. Use the Creator to create the html elements to use for your framework.
4. Create the Templates to build each html element
5. Create pages for your website using the element templates
6. Publish your website.

TreeView Commands

OneEvery commands are available in the TreeView to enable one to execute and work with OneEvery easily. These are New App, Open App, Publish App etc.

Commands

When using OneEvery, one is provided with buttons, property bags etc.



The above buttons are availble in most grids that you will find. These are

1.  Render elements to html previewer
2.  Sort the tab-order using the row position of the listed items.
3.  Delete all listed records

4.  Save all listed records

5.  Add a new record, a new empty record will be added to the grid with an id of '-1' and you can type text and select options for the record. This is inline text editing.

Inline Row Commands

Inline commands are record based per each record that is existing for the module you have selected. As an example, each line in the grid might have these Action buttons.



These can be explained as

1.  Save current row entry

2.  Delete current row entry

3.  Clone the current row entry i.e. create a duplicate of the current row

4.  Move up - move the row up, this changes the tab index

5.  Move down - move the row down

6.  Add child - add a child to the current row entry

7.  Wrap record - wrap the record inside a 'div'

Some row records might just have save, delete and clone commands, depending on the functionality being required to make that module to work.

Conventions Used.

Element - this will refer to a html element
Style - will refer to a css class and its attributes
Css - will refer to inline css for each element
TabIndex - this indicates the tab order of the item when being added
Active - when checked, each item will be rendered

Welcome

Welcome

Thank you for showing interest in OneEvery. This is an interesting project started on 05 January 2018, initialy as a learning curve into Bootstrap and then evolving simply and easily. As part of this App, a CLI (Command Line Interface) is being built to ensure a smooth learning curve on using the App to create BootStrap websites.

You will notice the use of App inside OneEvery, this are being used interchangeably.

Every details of a website/app in OneEvery is saved inside a SQLite database. This helps in differentiating the various projects that one can create. However the elements, tags, classes etc are saved in a universal SQLite database as these can be used across projects.

To create a website, one creates each html page needed for the website. In each web page, css files and js files should be linked. To create a website with OneEvery, you follow these steps.

1. Create the app
2. Add CSS Files
3. Add JavaScript Files
4. Create Pages
5. Add Elements to your pages
6. Publish your website.

You can also use the CLI (Command Line Interface) to do most of these functions.

What's new

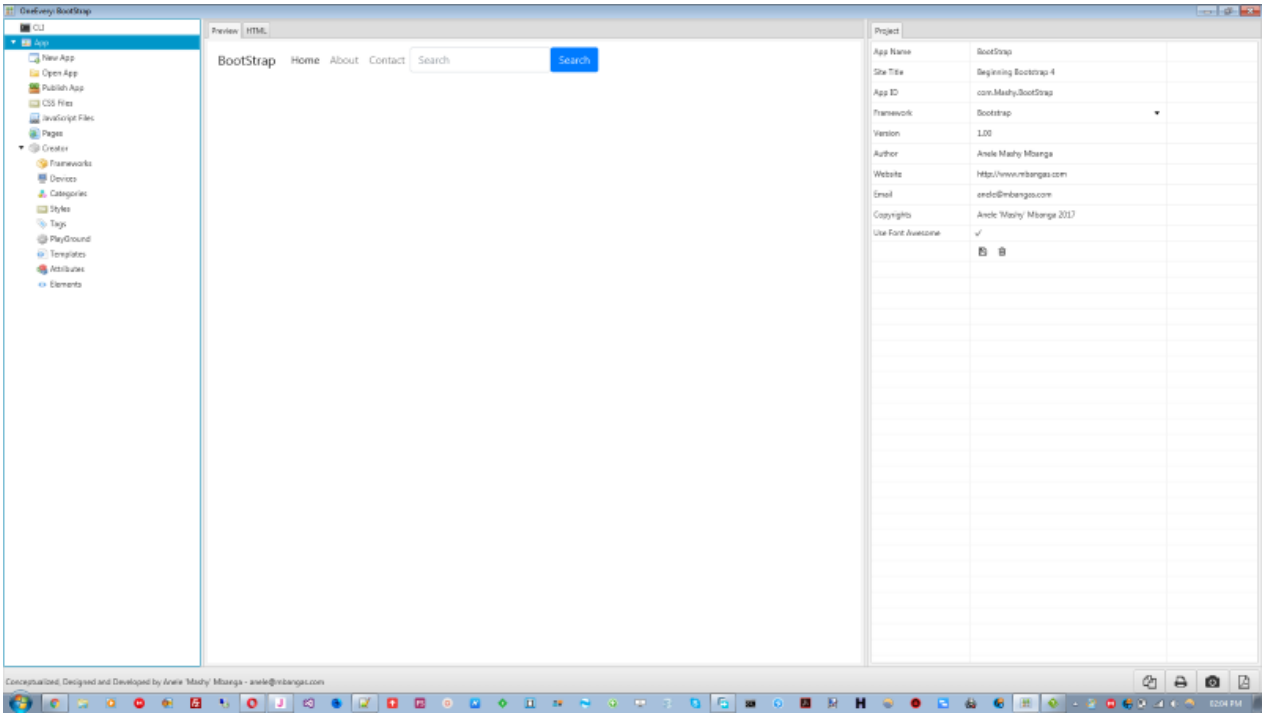
What's New

Getting Started

Getting Started

OneEvery's main screen is broken into sections in order to meet particular requirements. In most cases one is provided a

1. Treeview
2. Middle Part (html viewer, element, attribues and css) and
3. Property Bag

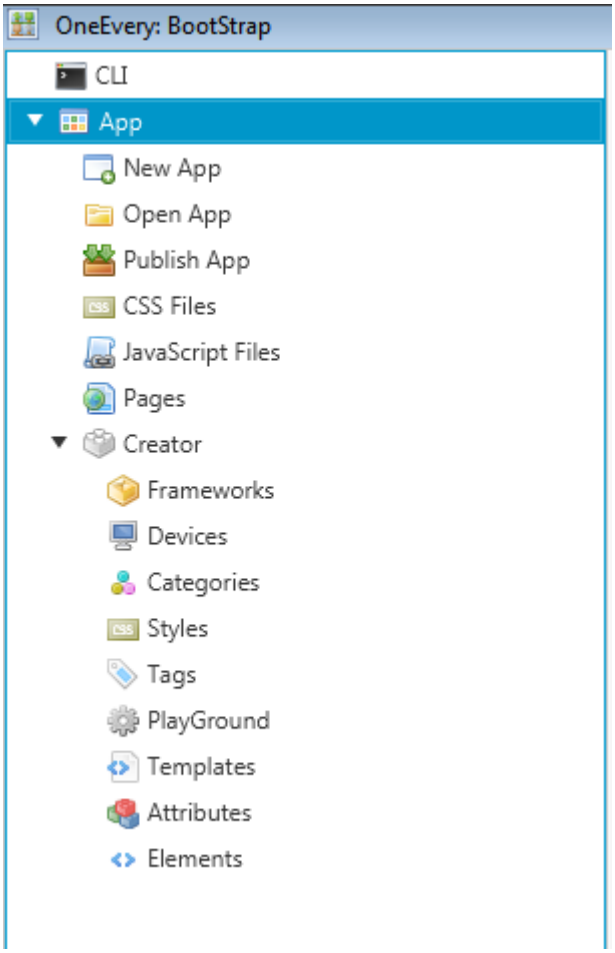


We will look at each of the TreeView commands below.

TreeView

TreeView

The TreeView provides a lot of functionality to use OneEvery and is the main point of call for everything OneEvery.



- **CLI**: Command Line Interface - here you type in commands to use when creating your website project.
- **App > New App**: here you create your new website app. This will ask you for the app name, app id, app title, version, author etc.
- **App > Open App**: here you are prompted to open an app project via the SQLite database.
- **App > Publish App**: here the website is compiled and saved in a folder for publishing.
- **App > CSS Files**: here the css files that will be used by the website are defined. The 'Tab Index' indicates the sequence of adding the css file links to each page.
- **App > JavaScript Files**: here the js files that will be used by the website are defined. The 'Tab Index' indicates the sequence of add the js files to each page.
- **App > Pages**: here these are the pages created for the website.
- **App > Creator**: here is the builder to knit everything else that will be used by the framework. An attempt will be made to test other frameworks and the starting point for that will be the Creator. This is broken into various commands for basic html element management.

App > Creator

- **Frameworks** - these are the various different frameworks that one can add
- **Devices** - these define the device sizes for each framework.
- **Categories** - each of these is defined from the (html) elements that can be created.
- **Styles** - these are css classes per each of the frameworks

- **Tags** - these are html tags e.g. a, p, h1 etc, these are also refresh-able from the (html) elements created.
- **PlayGround** - this provides functionality to 'test' your html code. You can copy and paste html code here to view it. This depends on the framework chosen and css and js files specified and active for that framework in use.
- **Templates** - these are (html) elements templates that one can build to facilitate page creation.
- **Attributes** - these are to build the (html) elements to be used when creating pages
- **Elements** - these work hand in hand with attributes to 'build' up the (html) elements to be used in creating pages

CLI

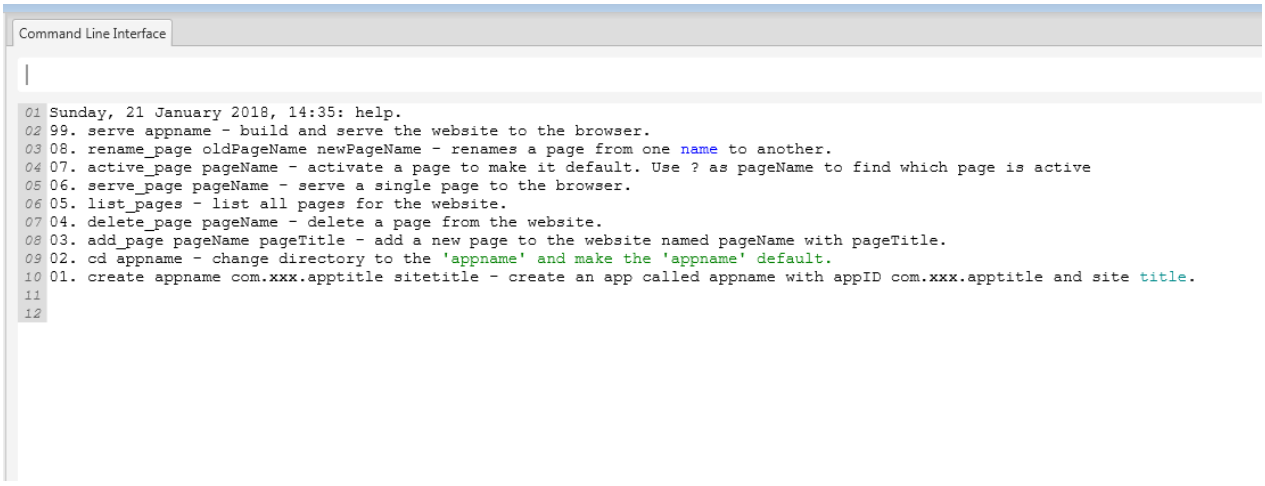
CLI

CLI stands for **C**ommand **L**ine **I**nterface. This is a 'shortcut' for most basic OneEvery functionality. As OneEvery grows, more commands will be added.

You activate the CLI by clicking the CLI treeitem and the CLI command will appear in the 'middle' portion of the screen. Here are some commands that you can execute. You just type a command in the command line and press the <Enter> key.
The commands are incremented with the last command and its result at the top of the command screen. Thus you dont have to scroll down to see the last commands entered and executed.

CLI Commands

help - *view all commands that you can execute. The commands will be listed from 01...n*



The commands help with repetitive commands that one will use when using OneEvery.

NB: The CLI Arguments should not have a space character

1. Create a new website app

create appName appID appTitle, e.g. *create contacts com.Mashy.Contacts Contacts*

This commands expects 3 arguments, the appName, appID and the appTitle. Once created, you can select the App command in the treeview to update the project details.

2. Activate a website app

You can activate any website app that you have created with the CD command. This activates the app so that one can work on it. With the example above, one would then type

cd contacts

3. Adding a page to the project.

Once the project is active, you can add a blank page to it by using the add_page command. This expects the pageName and pageTitle arguments

add_page pageName pageTitle e.g. ***add_page*** *addeditcontact Contacts*

4. Delete a page

You created a page and now wants to delete it. You can use the delete_page command. This expects the pageName to delete.

delete_page pageName e.g. **delete_page** addeditcontact

5. Rename a page

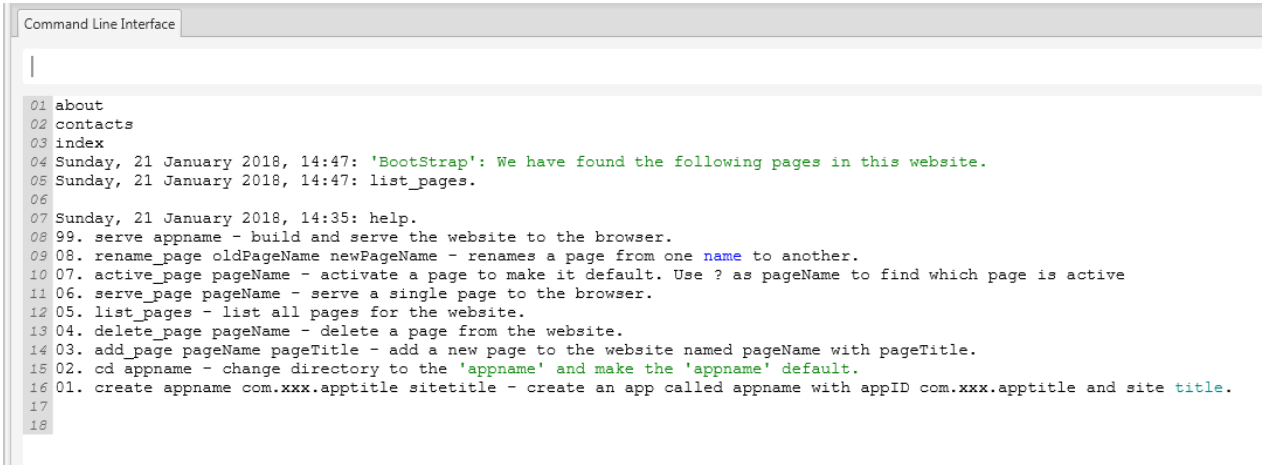
You created a page and now want to rename it. You can use the rename_page command. This expects the oldPageName and newPageName arguments

rename_page oldPageName newPageName e.g. **rename_page** addeditcontact contact

6. List all pages in the website

You can list all available pages for your website using the **list_pages** command. This does not expect any arguement

list_pages

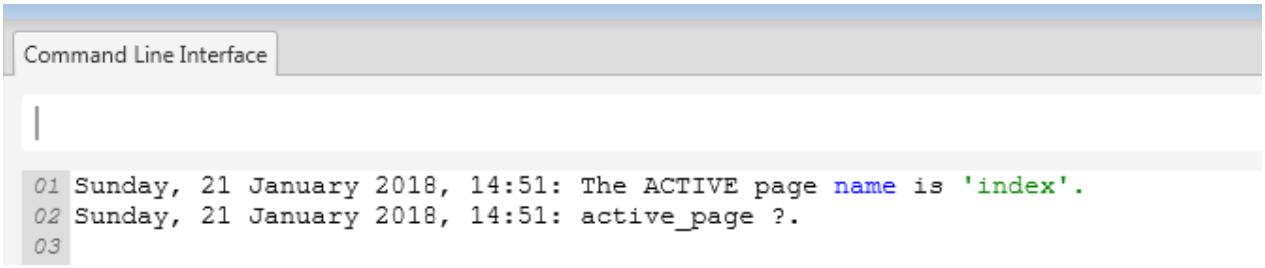


7. Activate a page / Which page is active?

Before you can add any components/elements in a page using the CLI you need to make the page active. This has the effect of ensuring that OneEvery remembers which page you are working on.

You can use the **active_page** pageName to set any page to be active specifying the page name, e.g. **active_page** addeditcontact

To know which page is active, you can use the active_page command with a question mark, e.g. **active_page ?**



8. Serve Page / Website (to browser)

Once you have built your page and thus website, you can serve it to a browser to view how it looks like. There are two commands provided for this. One is to serve a page and the other to serve the whole website. Serving builds up each of the affected pages and the complete website to the "Projects" folder for you.

8.1. Serve a Page

To serve a page that you are working on i.e. active_page, you can use the **serve_page** command e.g. **serve_page** addeditcontact

This will build the page and open it in the browser.

8.2. Serve the website

To serve the website to the browser, you can use the **serve** command with the appName e.g. **serve** contacts

App

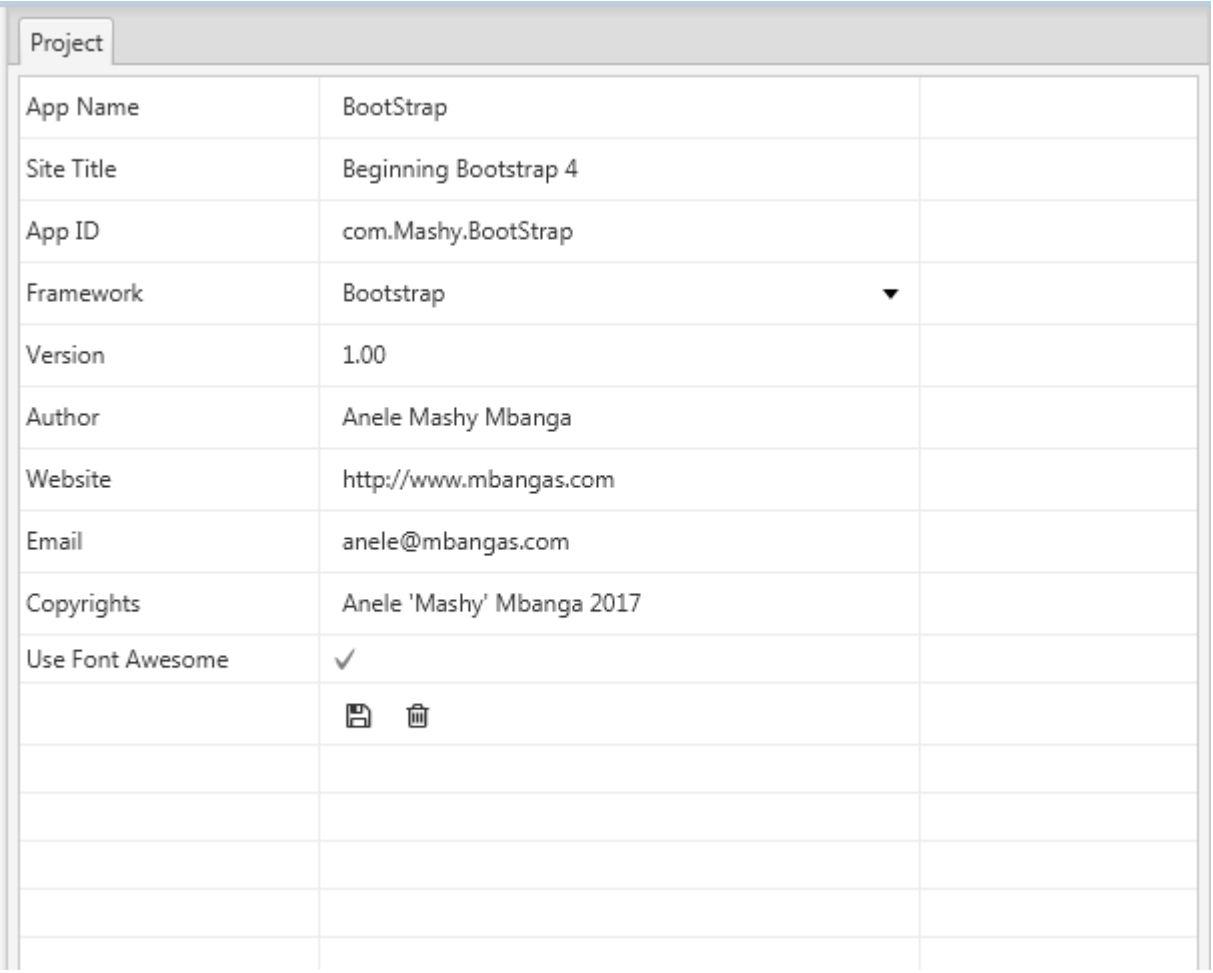
App

The App tree item, provides most functionality and commands that one can use in building their website. This is broken down into into two modes.

1. App functionality e.g. New App, Open App, Publish App, CSS Files, JavaScript Files and Pages
2. Creator - this is used to define the (html) elements to be used in your app.

Remember, as there is no hard-coded elements in OneEvery, one has to build the elements to use, making OneEvery very flexible. Whilst OneEvery will come with Bootstrap 4, other frameworks can be added easily.

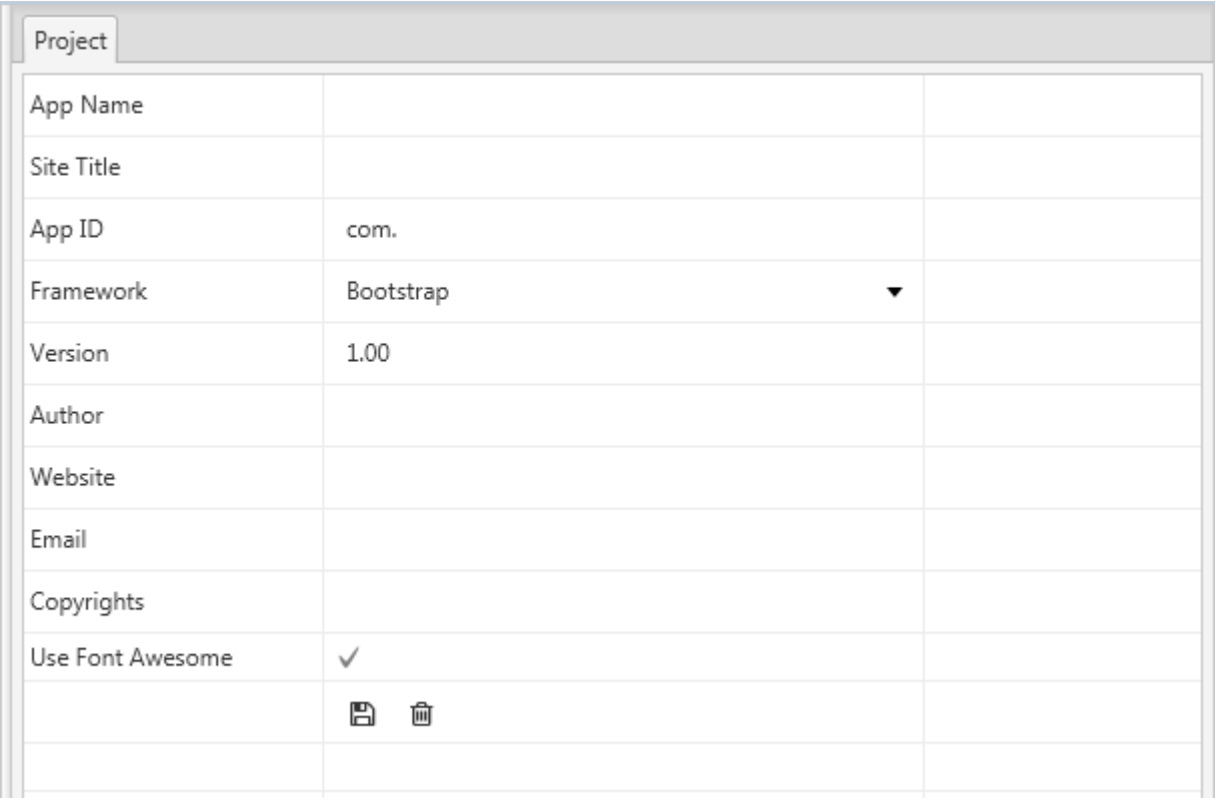
Selecting the **App** command in the tree activates the active project and you can update its details.



New App

App > New App

To create a new app/website using OneEvery tree command, select the '**New App**' command a property bag for the project details will be availed in the right section of the screen.



Enter the appName (no spaces), appTitle (Site Title), appID, framework, version etc. When done, select the save button.

You can create an app via the CLI and then update these details by selecting the **App** command. To create an app via the CLI, you type create appName appID and appTitle. For more details see the CLI.

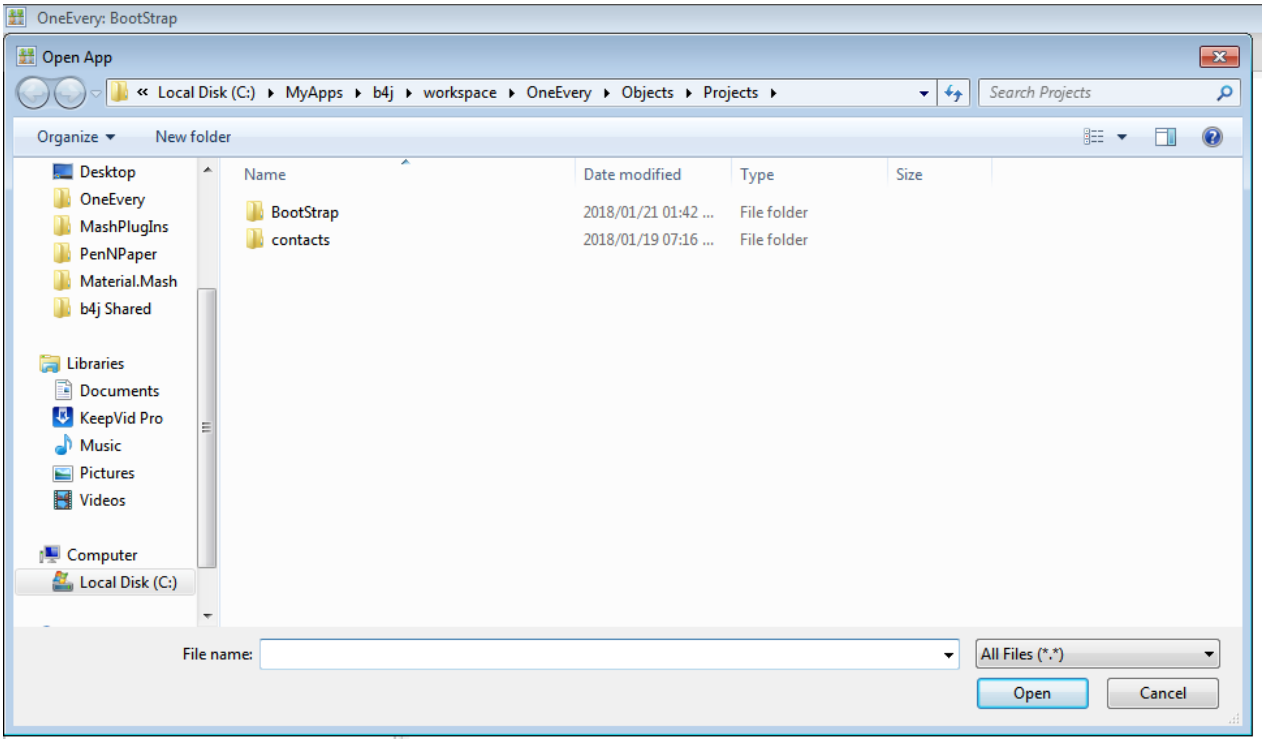
Each project created in OneEvery is saved under the **Objects\Projects\appName** folder. This is the same location where the projects are published to.

Open App

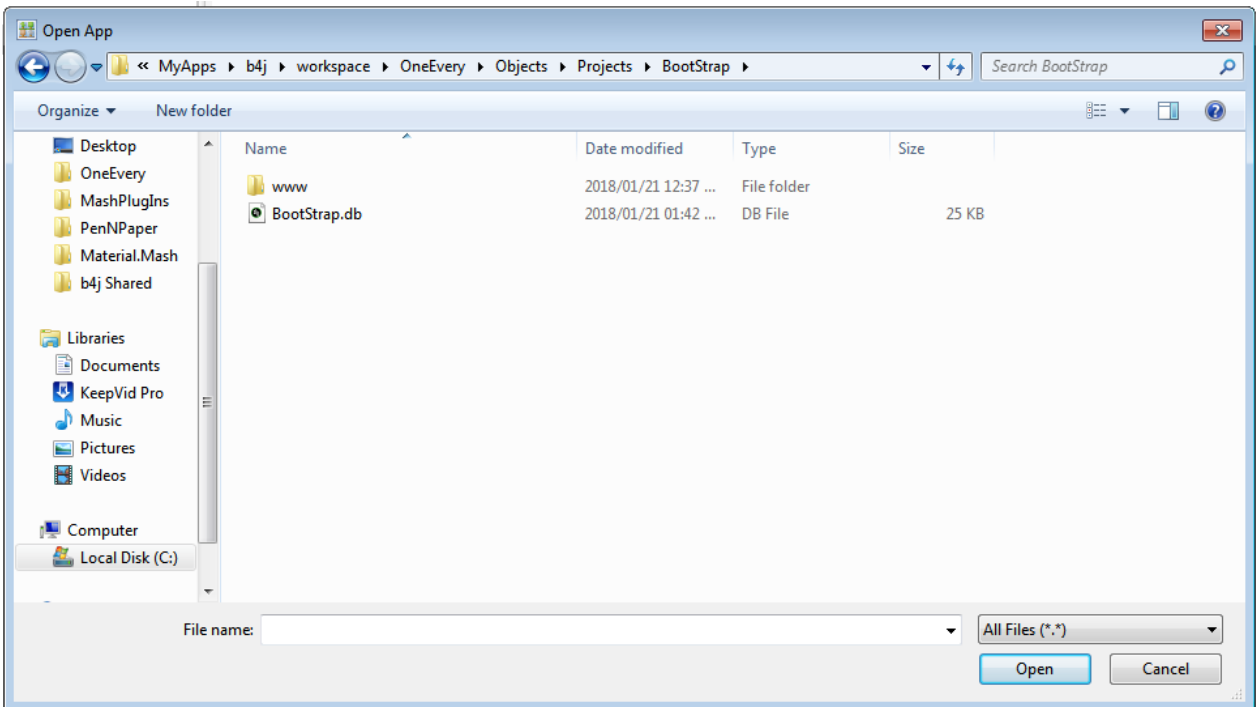
App > Open App

You can open an existing OneEvery app that was previsouly created by using the **Open App** command in the tree.

Below are two project folders, you will double click on the project folder, e.g. Bootstrap and then select the Bootstrap.db file to open the project.



Select Project App Database e.g. **BootStrap.db**



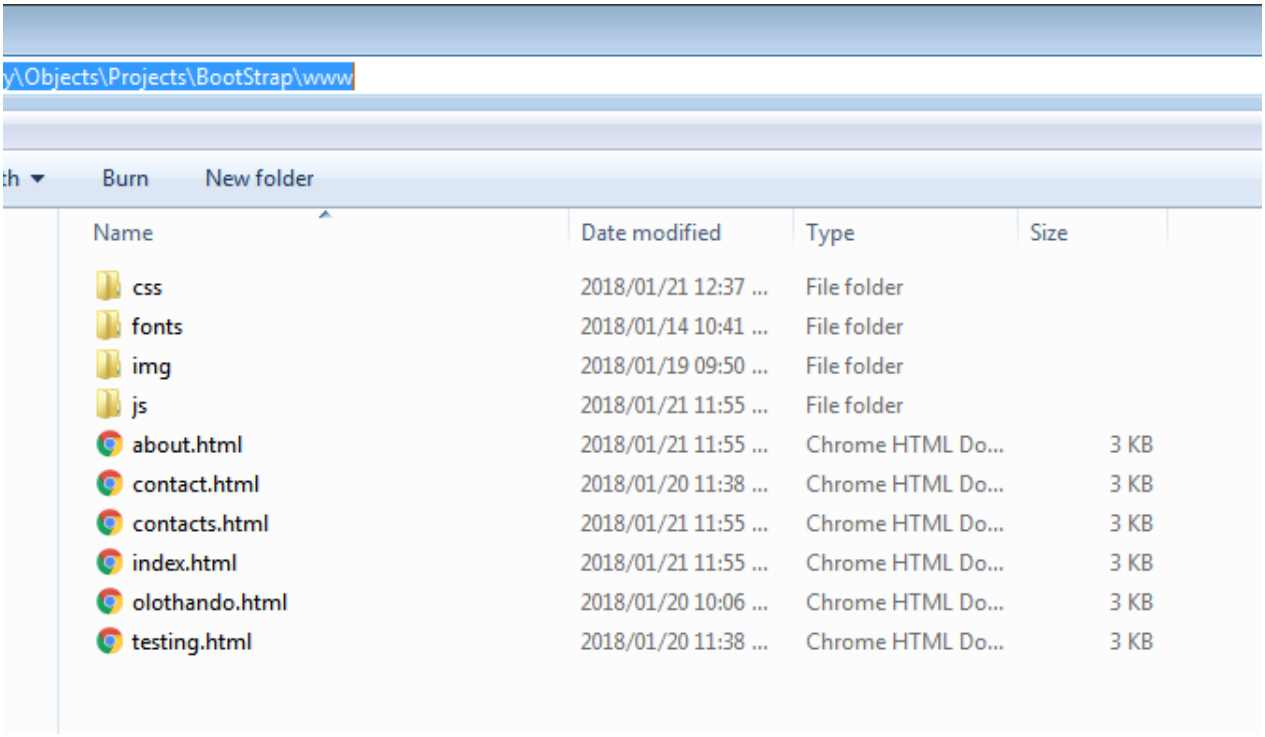
This is the same way you will open your website project. The same process here is the same as running the **cd bootstrap** command in the CLI, so instead of clicking Open App and selecting these folders, you can easily run the cd command.

Publish App

App > Publish App

The **Publish App** command has the same effect of building all your website pages and then serving them to the browser.

This is the same as running **serve** appName in the CLI.

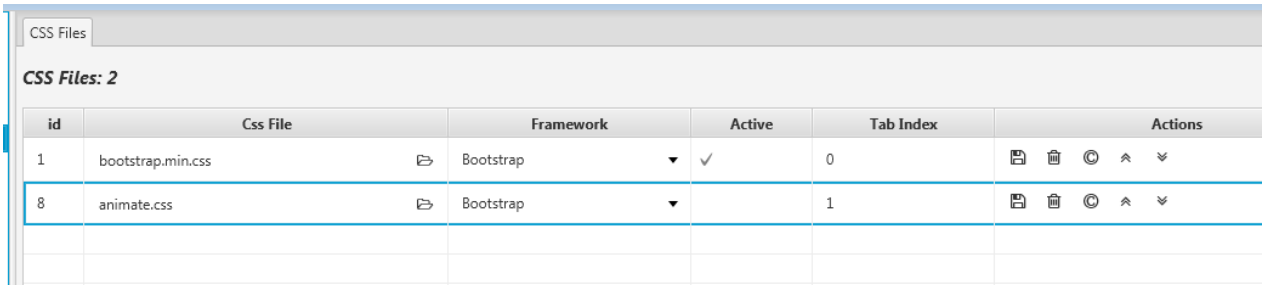


As noted above, the published project is saved in **Objects\Projects\appName\www** folder. This is the folder that you can copy to your server.

CSS Files

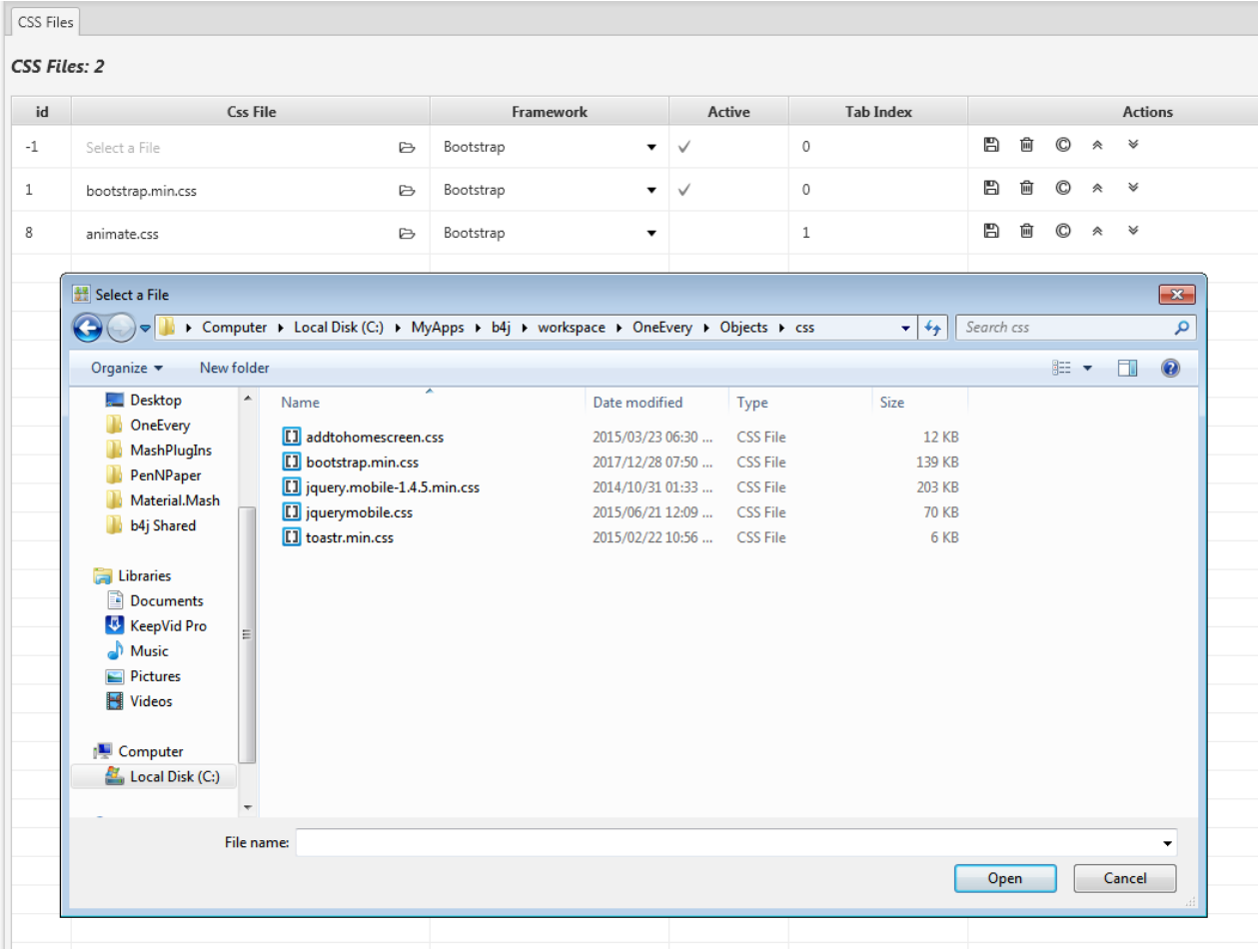
App > CSS Files

This is where one defines the CSS files that the website will use. All CSS Files defined here will be used in all the pages of the website.



As noted above, each css file to be used is defined and its 'tab index' i.e. sort order is specified. From above, tabindex 0 will be added first before animate.css

- To add a css file, click the Add button.  , an empty row will be added as the first entry with -1 as the id.
- Click the folder icon and select in the file chooser



- Select the framework the css file will be applicable to.
- To be used in your app, the css file should be active.
- To indicate the sort order / adding order of the css file to each of the pages, change and update the tab index property.
- Click save to update the row entry

Commands

1. Save current row entry

2. Delete current row entry

3. Clone the current row entry i.e. create a duplicate of the current row

4. Move up - move the row up, this changes the tab index

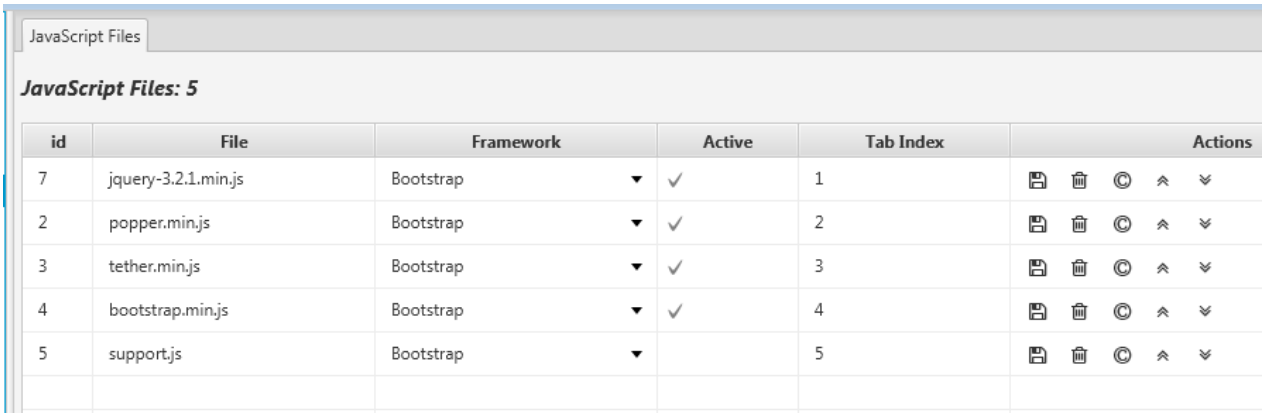
5. Move down - move the row down

JavaScript Files

App > JavaScript Files

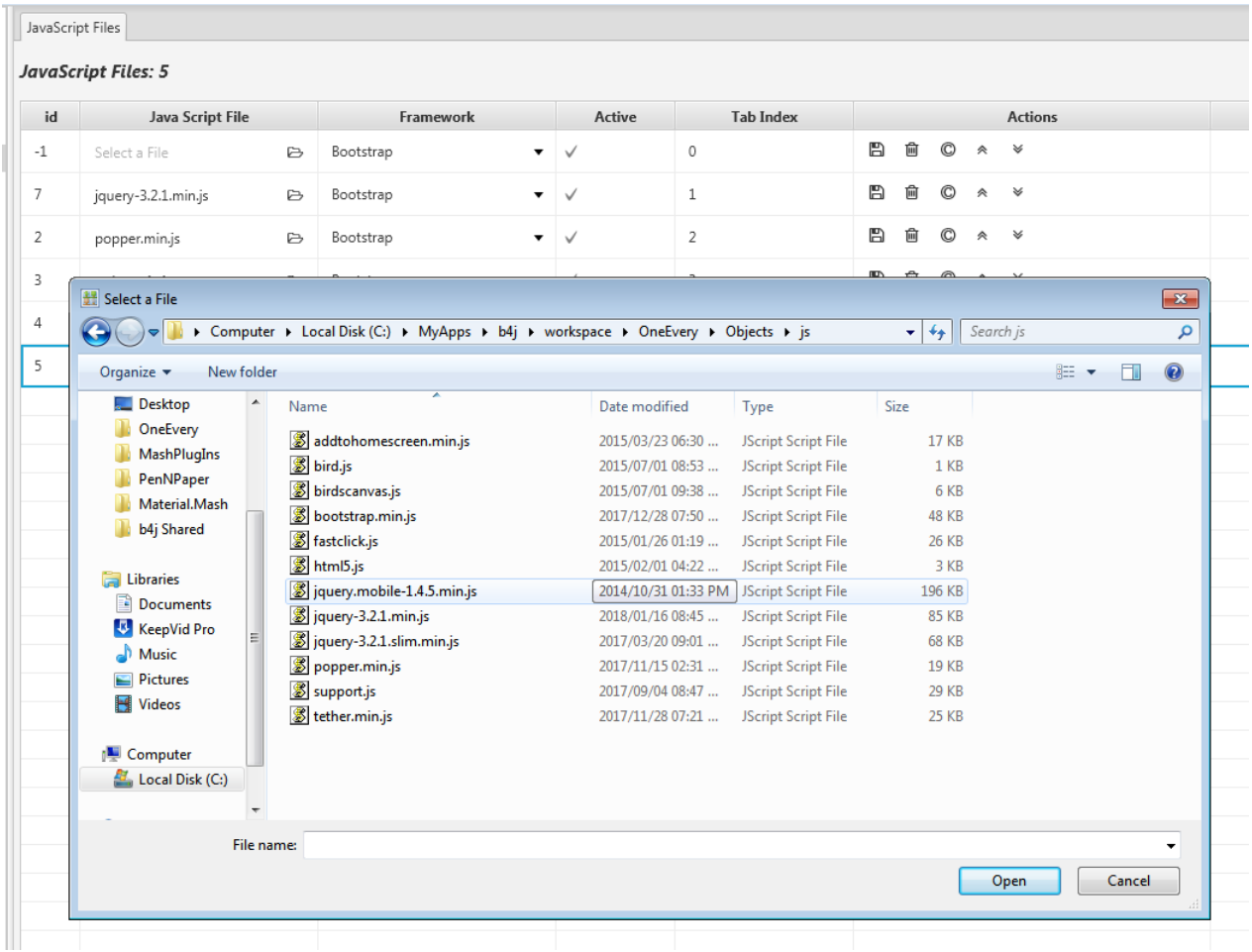
This is where one defines the JavaScript files that the website will use. All JavaScript Files defined here will

be used in all the pages of the website.



As noted above, each js files to be used is defined and its 'tab index' i.e. sort order is specified. From above, files will be be added from tabindex 1 to 5 in sequence.

- To add a js file, click the Add button.  , an empty row will be added as the first entry with -1 as the id.
- Click the folder icon and select in the file chooser



- Select the framework the js file will be applicable to.
- To be used in your app, the js file should be active.
- To indicate the sort order / adding order of the js file to each of the pages, change and update the tab index property.
- Click save to update the row entry

Commands

1. Save current row entry

2. Delete current row entry

3. Clone the current row entry i.e. create a duplicate of the current row

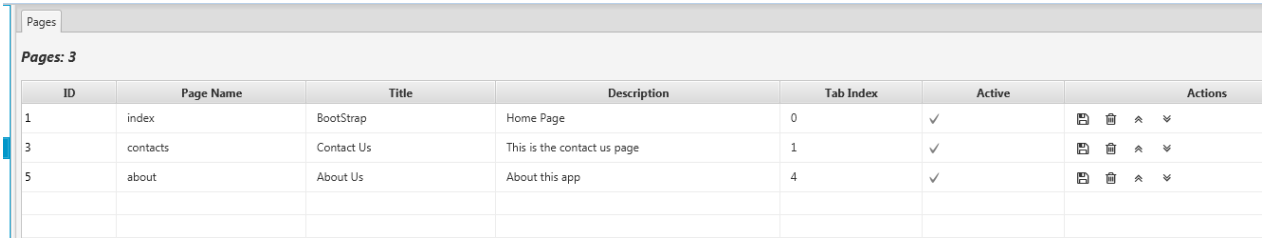
4. Move up - move the row up, this changes the tab index

5. Move down - move the row down

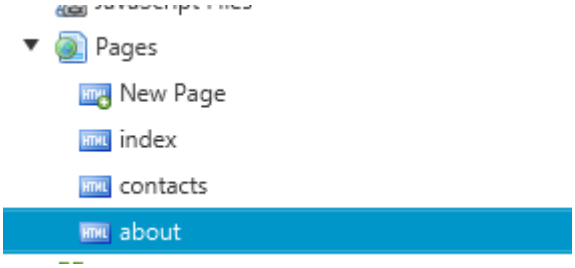
Pages

App > Pages

The Pages command allows one to create all the pages for their website. Below is a list of pages that have been created. You can use the CLI to create a page too.



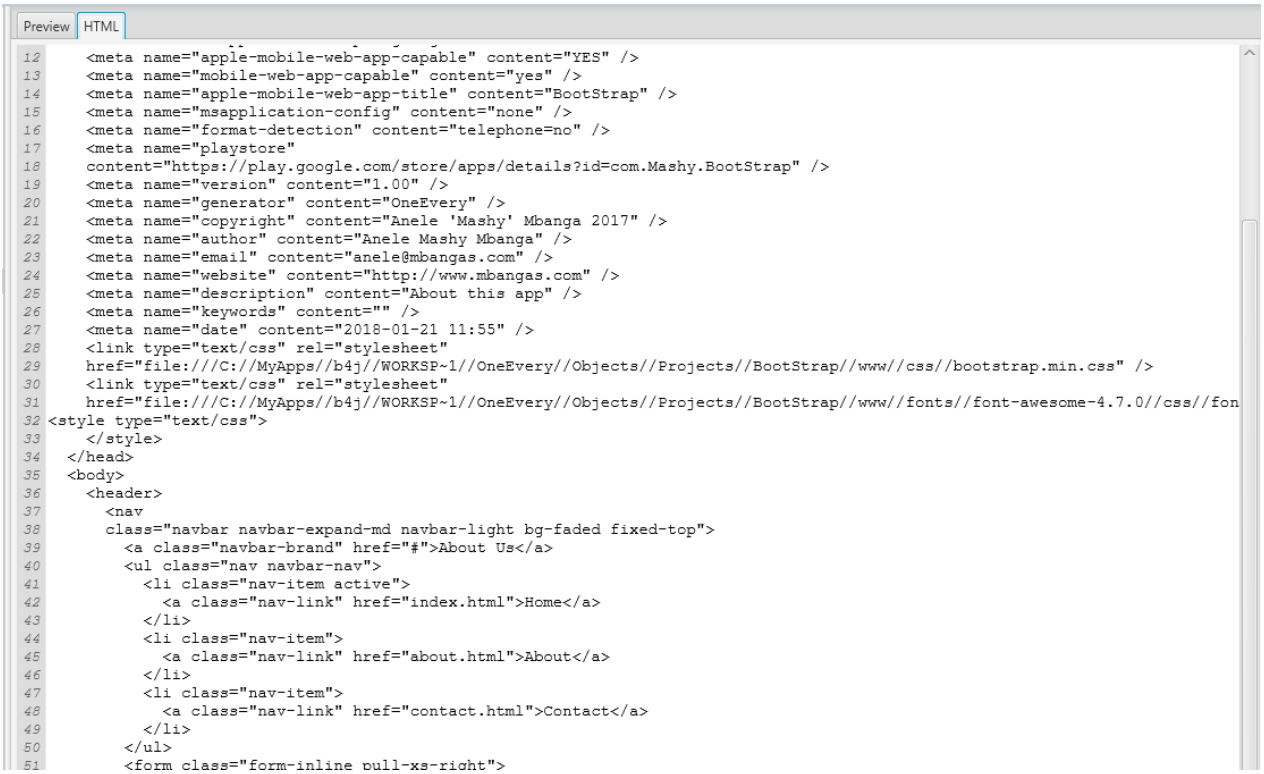
When the Pages command is selected, all pages in the website are also loaded in the tree view and one can access each page by selecting it in the treeview. Selecting a page in the treeview enables one to also preview it.



Example Page: About Page



Example Page: The page above is an example of a page that one can Preview and also view the HTML code for it..



As noted above, as you build your page, you are able to both preview the page look and feel and the also view its html code.

Commands

1.  Save current row entry

2.  Delete current row entry

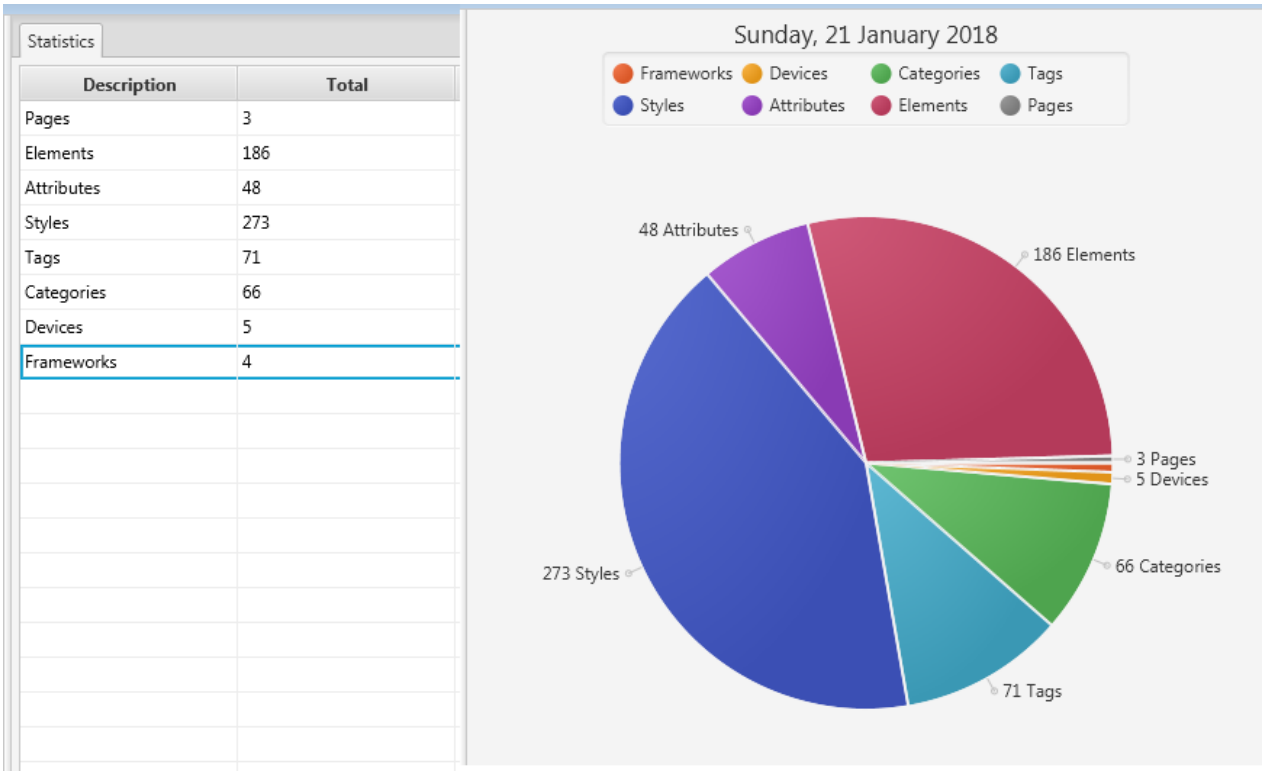
3.  Move up - move the row up, this changes the tab index

4.  Move down - move the row down

Creator

App > Creator

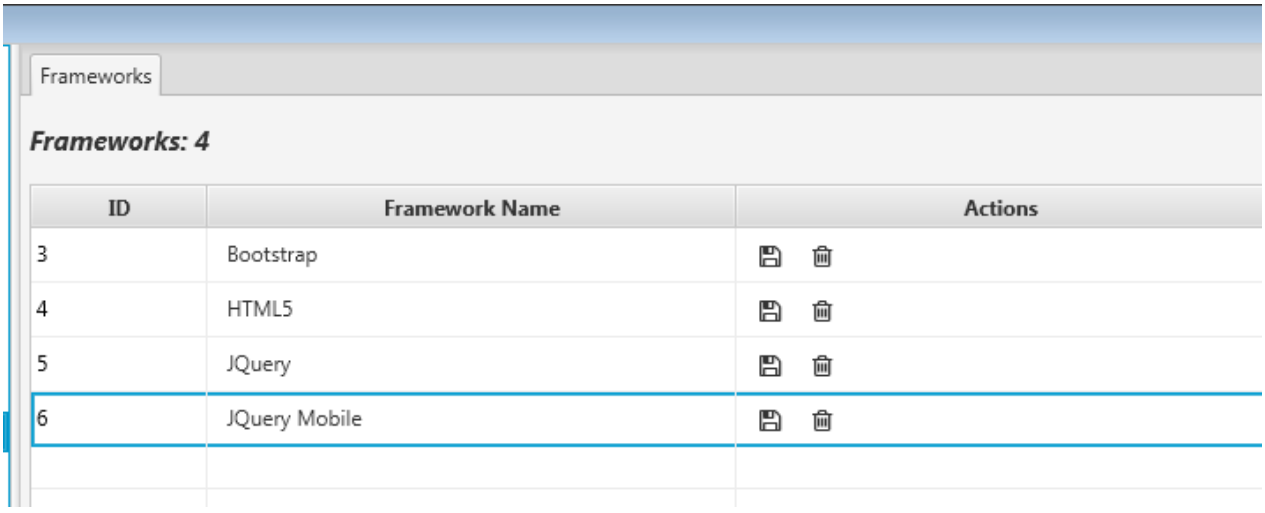
The Creator enables one to build their framework. The current framework that comes with OneEvery is BootStrap 4. Some of the modules above are generated based on the available elements that are already defined in OneEvery.



When selected, the Creator command provides statistics about what is happening inside OneEvery. As on 21 January 2018, there were 3 pages, 186 elements, 48 attributes, 273 style classes etc etc, based on the elements that have been created.
This content is depended on the treeitems below the Creator Command.

Frameworks

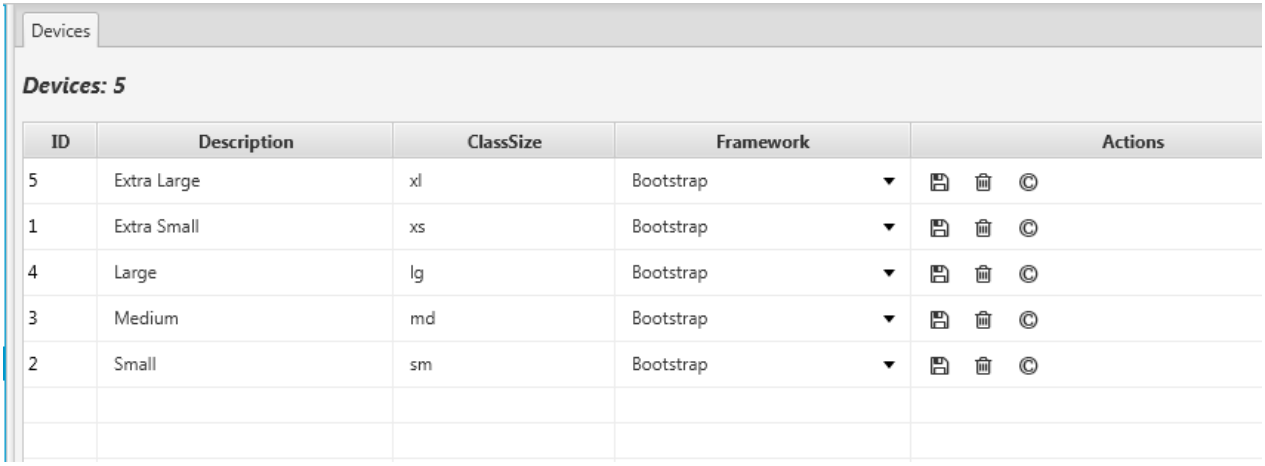
App > Creator > Frameworks



This is a list of frameworks that you can also add elements for in OneEvery. You can add more frameworks as you wish. OneEvery comes built with BootStrap 4 which is based on HTML5 using JQuery too.

Devices

App > Creator > Devices



Devices

Devices: 5

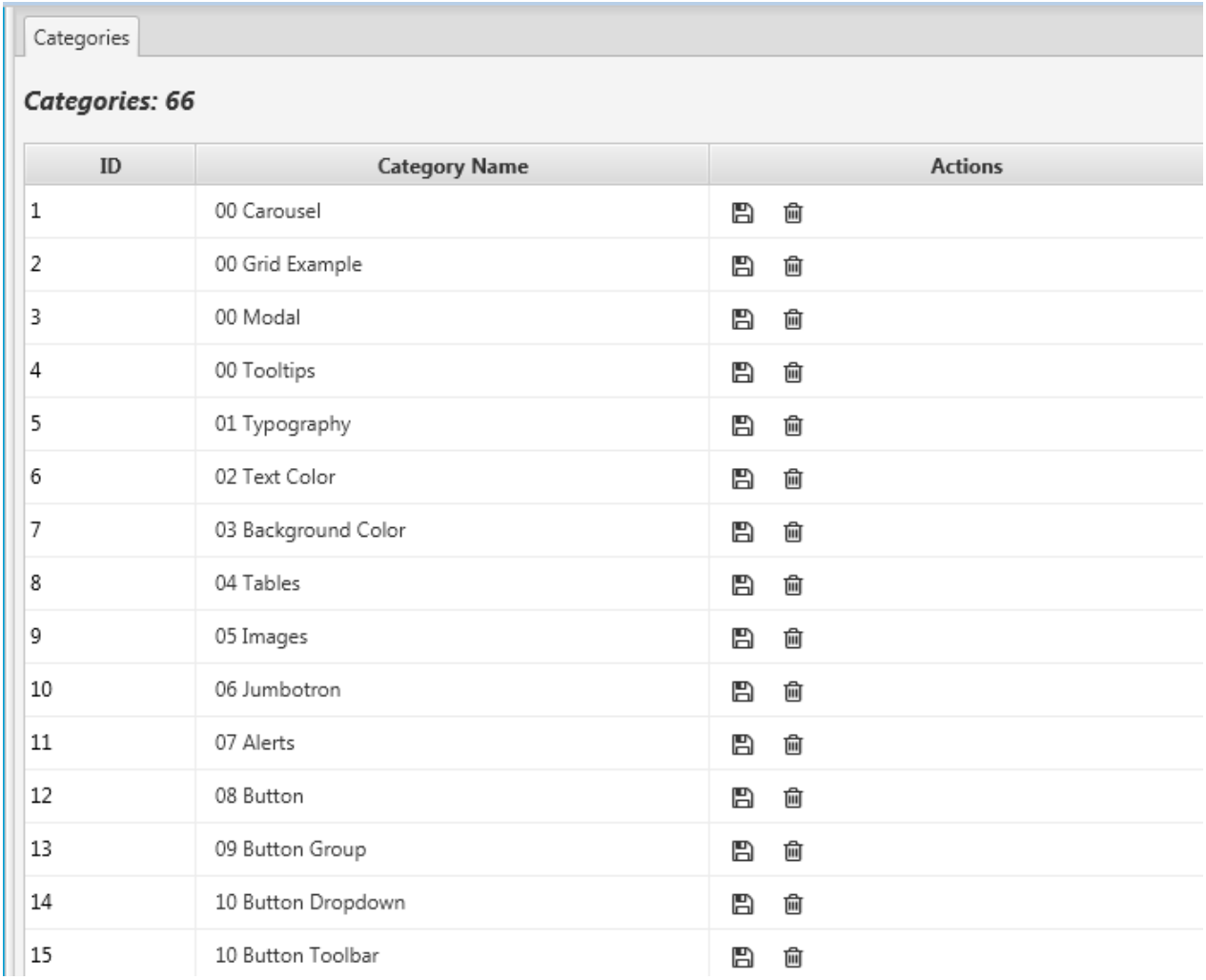
ID	Description	ClassSize	Framework	Actions
5	Extra Large	xl	Bootstrap	
1	Extra Small	xs	Bootstrap	
4	Large	lg	Bootstrap	
3	Medium	md	Bootstrap	
2	Small	sm	Bootstrap	

This is a list of devices and their ClassSizes for the bootstrap framework. These sizes are useful in defining the bootstrap grid and some other attributes and classes.

Categories

App > Creator > Categories

Categories are defined from elements. Each element created needs to be categorized for ease of identification. You will see on elements how this is linked.



Categories

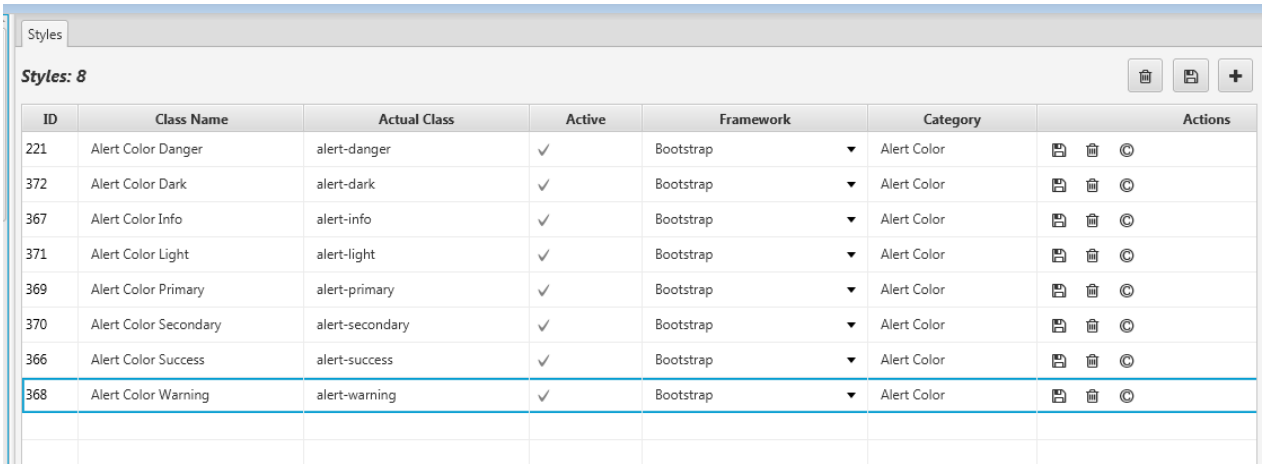
Categories: 66

ID	Category Name	Actions
1	00 Carousel	
2	00 Grid Example	
3	00 Modal	
4	00 Tooltips	
5	01 Typography	
6	02 Text Color	
7	03 Background Color	
8	04 Tables	
9	05 Images	
10	06 Jumbotron	
11	07 Alerts	
12	08 Button	
13	09 Button Group	
14	10 Button Dropdown	
15	10 Button Toolbar	

Styles

App > Creator > Styles

Styles are style class definitions used by the frameworks. These below are Alert Color classes.

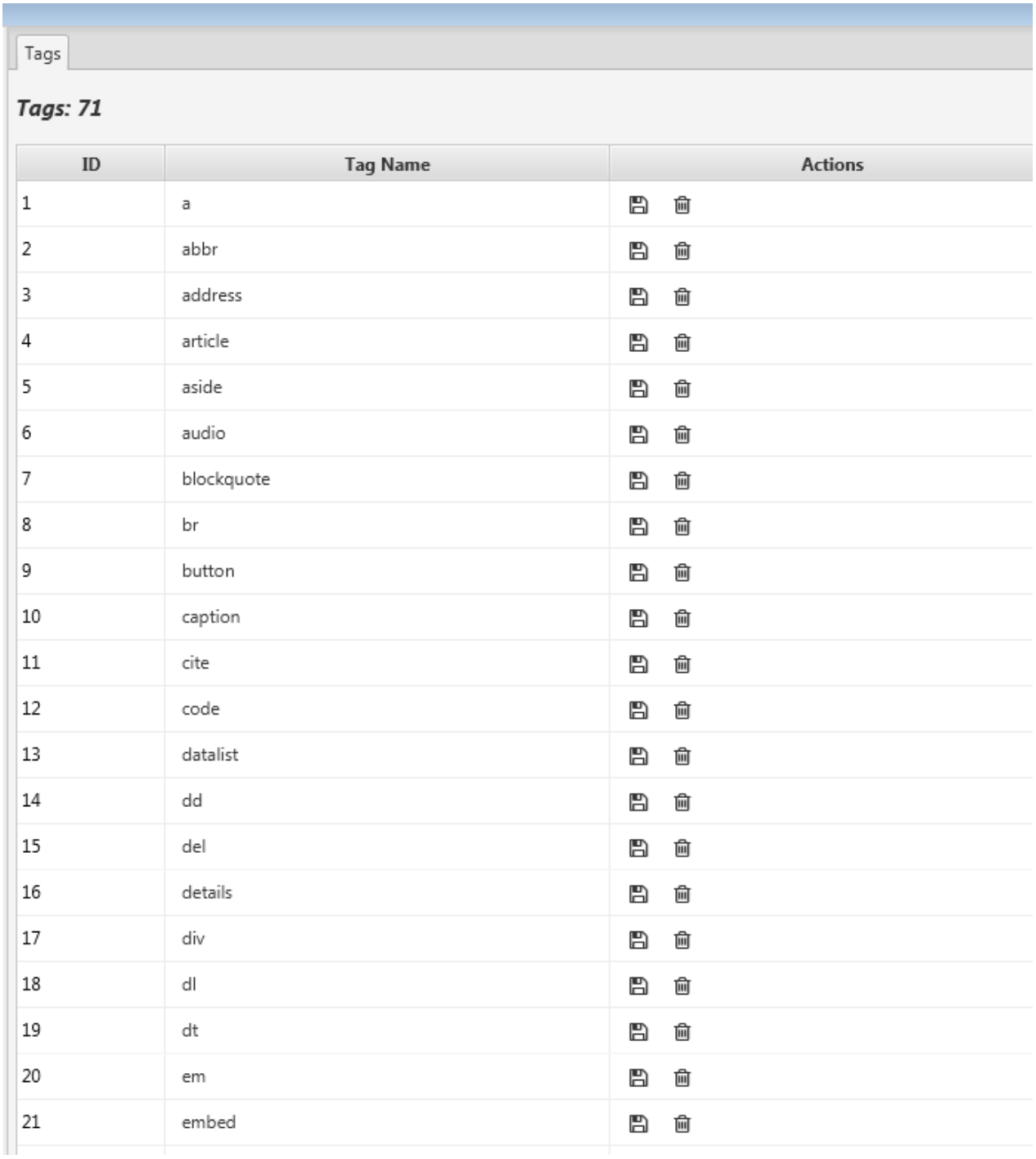


ID	Class Name	Actual Class	Active	Framework	Category	Actions
221	Alert Color Danger	alert-danger	✓	Bootstrap	Alert Color	
372	Alert Color Dark	alert-dark	✓	Bootstrap	Alert Color	
367	Alert Color Info	alert-info	✓	Bootstrap	Alert Color	
371	Alert Color Light	alert-light	✓	Bootstrap	Alert Color	
369	Alert Color Primary	alert-primary	✓	Bootstrap	Alert Color	
370	Alert Color Secondary	alert-secondary	✓	Bootstrap	Alert Color	
366	Alert Color Success	alert-success	✓	Bootstrap	Alert Color	
368	Alert Color Warning	alert-warning	✓	Bootstrap	Alert Color	

Tags

App > Creator > Tags

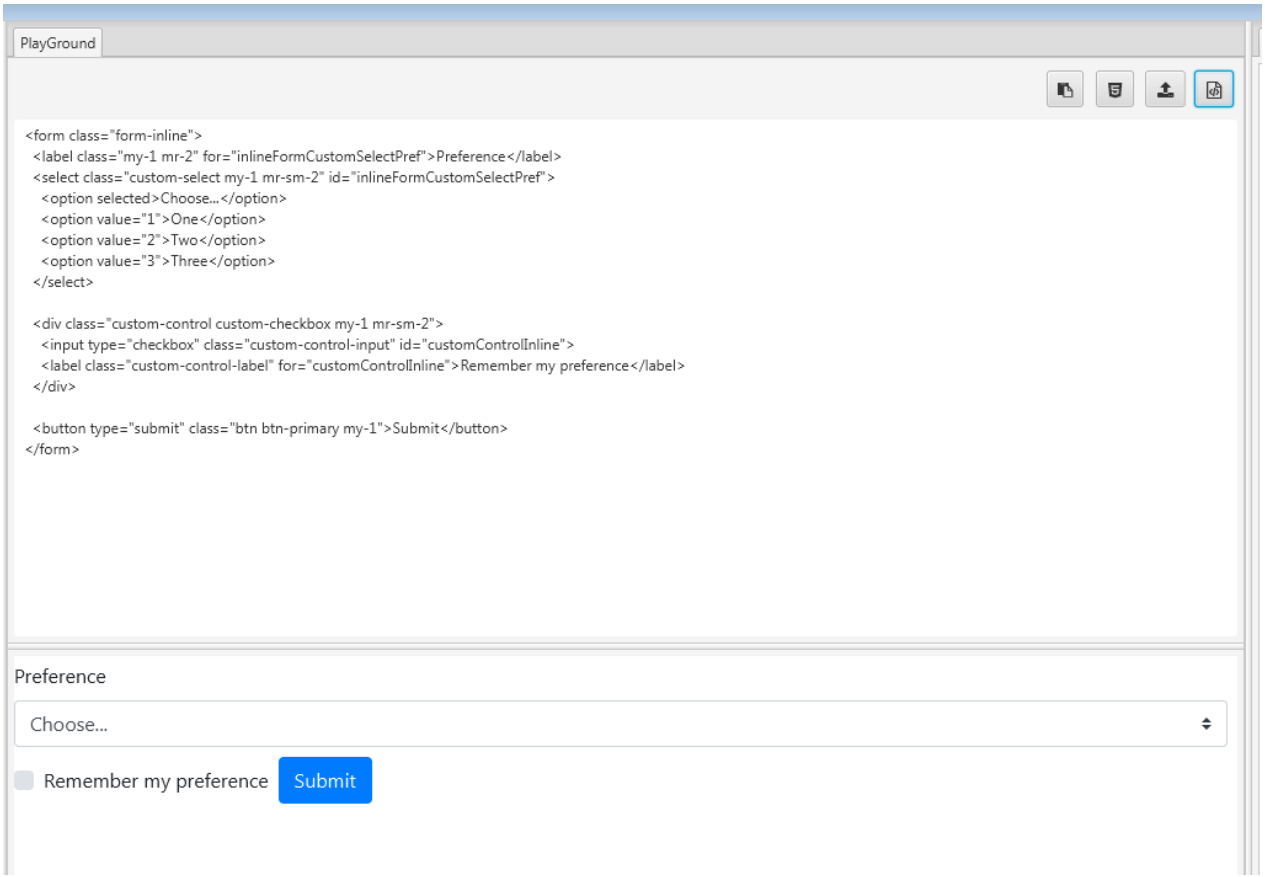
Tags are the html tags available to create websites. The ones listed here can be refreshed based on the elements that have been added. Most of what is available is HTML5 specific and is the basis for creating the elements and templates to build the websites.



PlayGround

App > Creator > PlayGround

The PlayGround allows one to play around with html code and also provides the ability to import the build code of the pasted html code inside OneEvery. This in essense is an easy way to create elements that one can use.
When playing around with html code, one needs to ensure that the linked CSS and JS Files are active in the project.



To use the PlayGround, one pastes HTML code into the upper section, then renders it on the bottom part to view how it will look like when executed. When rendered, the PlayGroud tree items in the treeview are also updated to reflect the structure.

1.  click here to paste html code from the clipboard.

2.  paste the last used html code to the area

3.  import elements to OneEvery. You will be prompted for the category to add to.

4.  parse the html code and render it for preview

You can paste and parse any html code in the playground, provided the css and js files are linked to execute it. The imported elements are accesible in the Attributes and Element commands. You will have update the Text property of the elements as these can be duplicated.

Templates

App > Creator > Templates

Templates provides html element templates that are built and usable to create page components.

Attributes

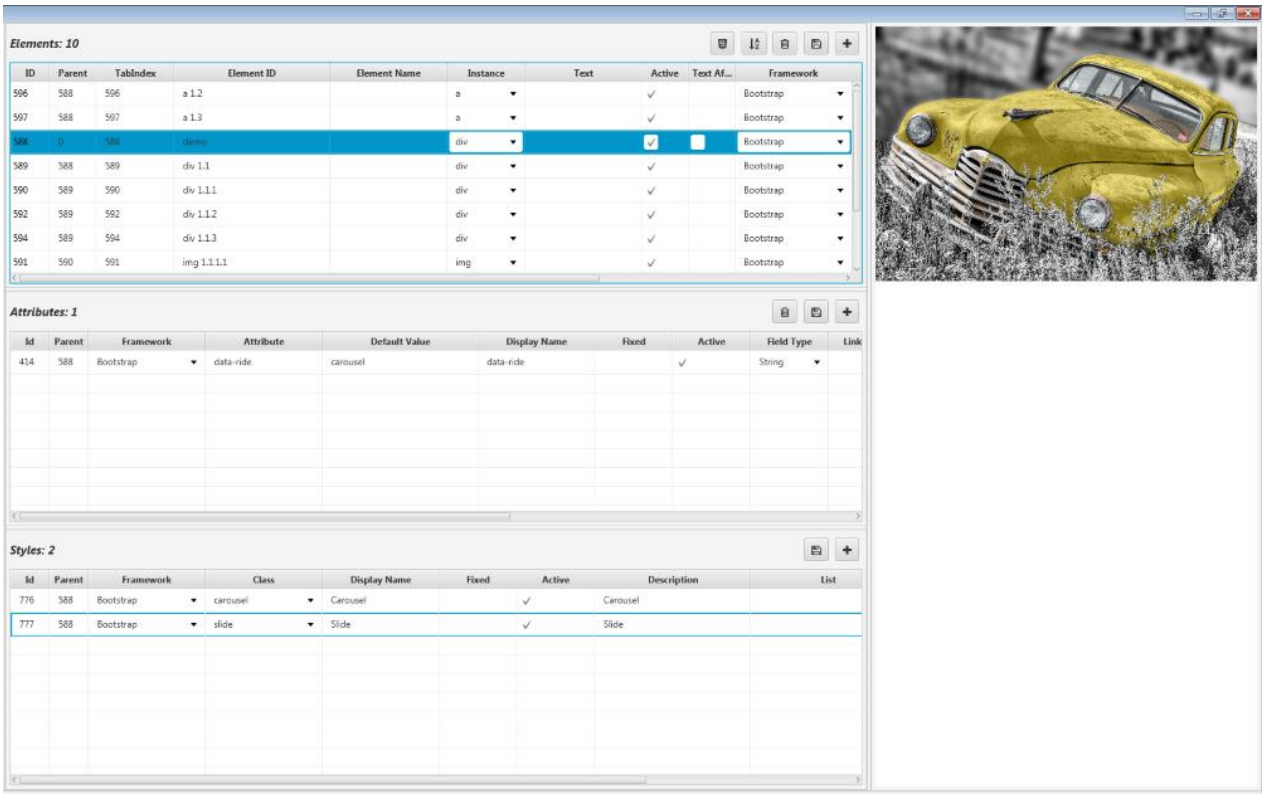
App > Creator > Attributes

The attributes command avails functionality to create elements including their styles (classes) , attributes and (inline) css for each element.

Each html element has attributes and a style. An element can also have child htmlelements, thus the **parent** property here. Each element in OneEvery is given a unique id number.
Level 1 elements will have Parent 0 while Level 2 elements will have parent ids other than zero. For example element 596 is a child of 588 whilst 588 has a zero parent, meaning its the initial element.
On the right section of the screen is the rendering of this zero (0) element.

On this screen there are 4 sections, the first is the element definition, the second attributes and the third css classes and the fourth, the preview of the element. Selecting an element will load each elements attributes and that elements css classes.
You can use the buttons to add, delete and update both the attributes and style classes.

Select the element first then one is able to add the elements and the styles.



To see the underlying html code for this element, select the Elements command in the creator.

Elements

App > Creator > Elements

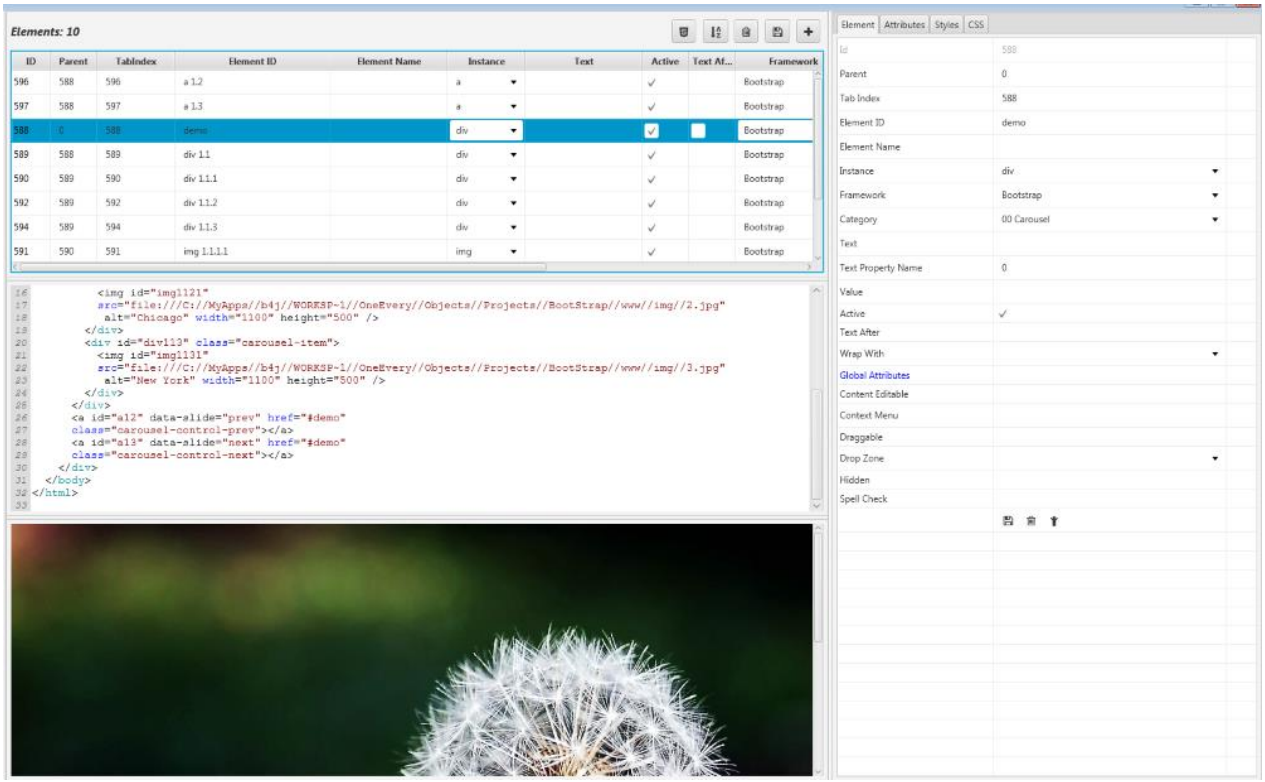
The elements command avails functionality to create elements including their styles (classes) , attributes and (inline) css for each element.

Each html element has attributes and a style. An element can also have child html elements, thus the **parent** property here. Each element in OneEvery is given a unique id number.
Level 1 elements will have Parent 0 while Level 2 elements will have parent ids other than zero. For example element 596 is a child of 588 whilst 588 has a zero parent, meaning its the initial element.
On the right section of the screen is the rendering of this zero (0) element.

On this screen there are 4 sections, the first is the element definition, the second html code and the third the preview and the right section a property bag just like one for the project. Selecting an element will load each elements attributes and that elements css classes. You can use the buttons to add, delete and update

both the attributes and style classes.

Select the element first then one is able to add the elements and the styles.



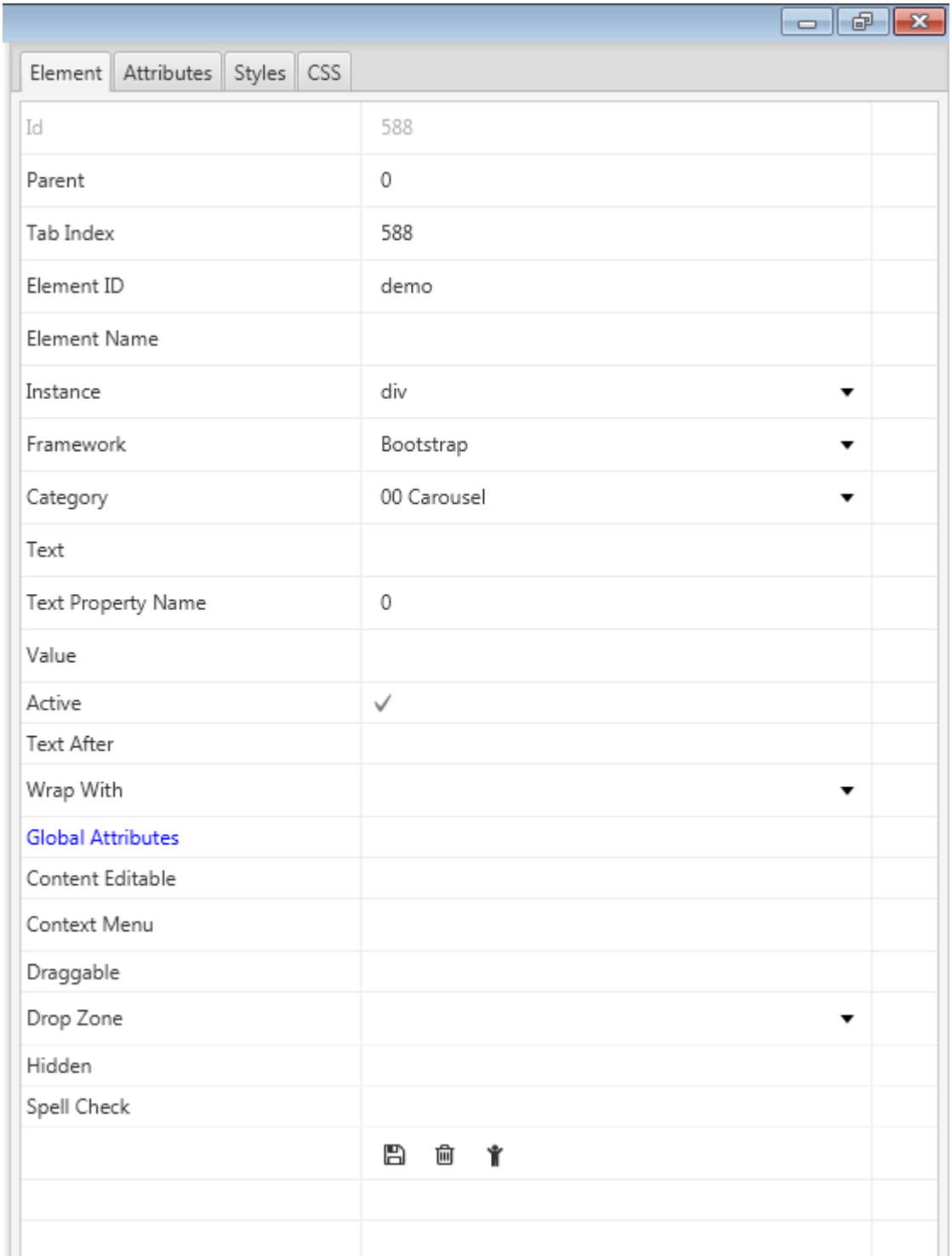
From above, each element selected in the first section will load the property bags that will have attributes, styles, css etc.

Element

App > Creator > Elements > Element Property Bag

The element property bag provides another way to update the element details linked to the grid. For the element to be rendered, it needs to be active. As per html elements, these have Text and Value, as indicated here.
As noted, this element is an instance of a 'div' based on html tags. The tab index also indicates the position of the element within its parent.

As you can see, the Category here is what appears in Categories and this is defined here.



You can save, delete and add a child to the element also within the property bag.

Attributes

App > Creator > Elements > Attributes Property Bag

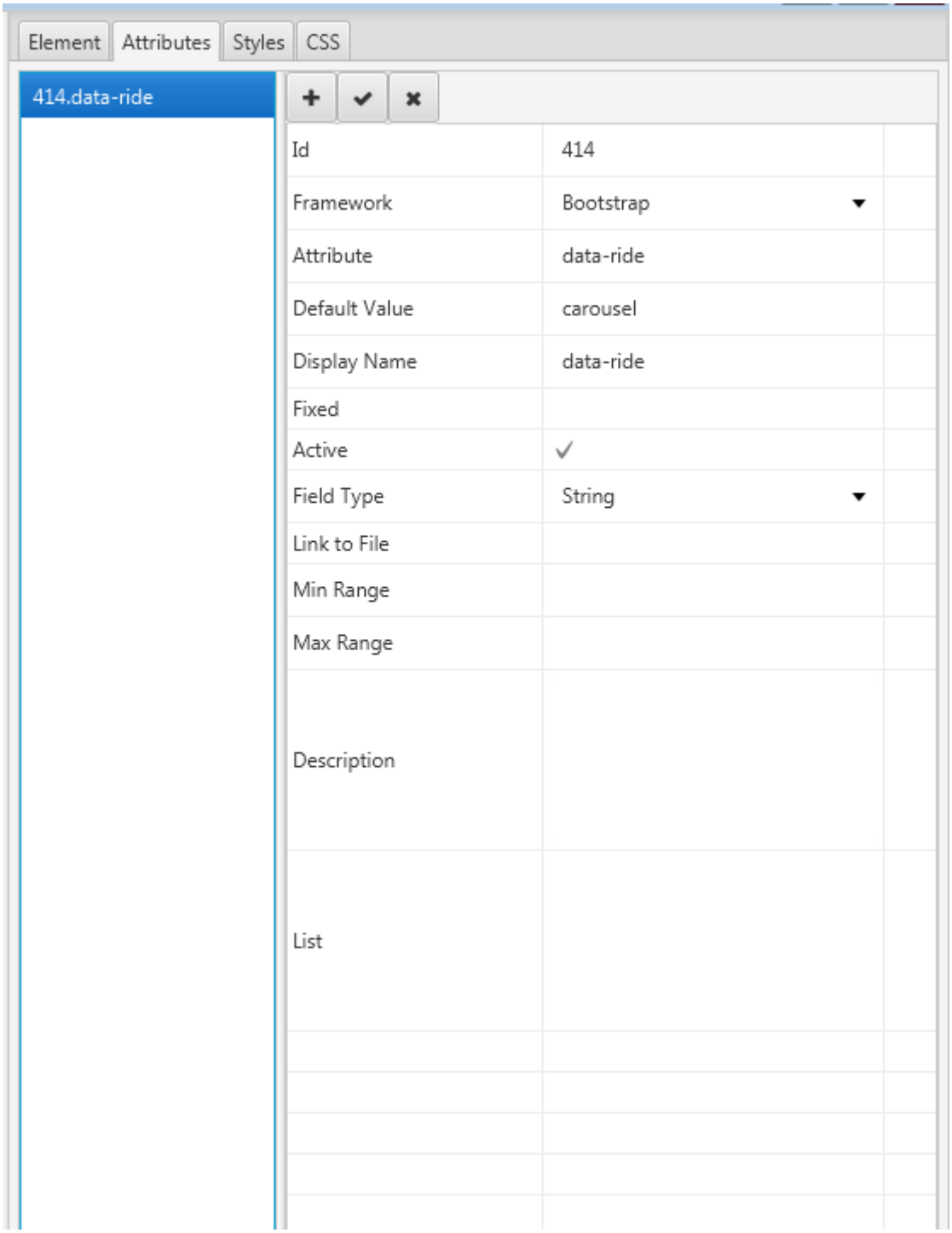
The element attributes as seen in the Attributes screen can also be defined here. For this element, a data-ride attribute has been added with the various properties.

To add a new attribute, click the add button

1.  Add attribute

2.  Save attribute (after capturing the properties)

3.  Delete attribute


Element
Attributes
Styles
CSS
414.data-ride
Id
414
Framework
Bootstrap
Attribute
data-ride
Default Value
carousel
Display Name
data-ride
Fixed
Active
Field Type
String
Link to File
Min Range
Max Range
Description
List

Styles

App > Creator > Elements > Styles Property Bag

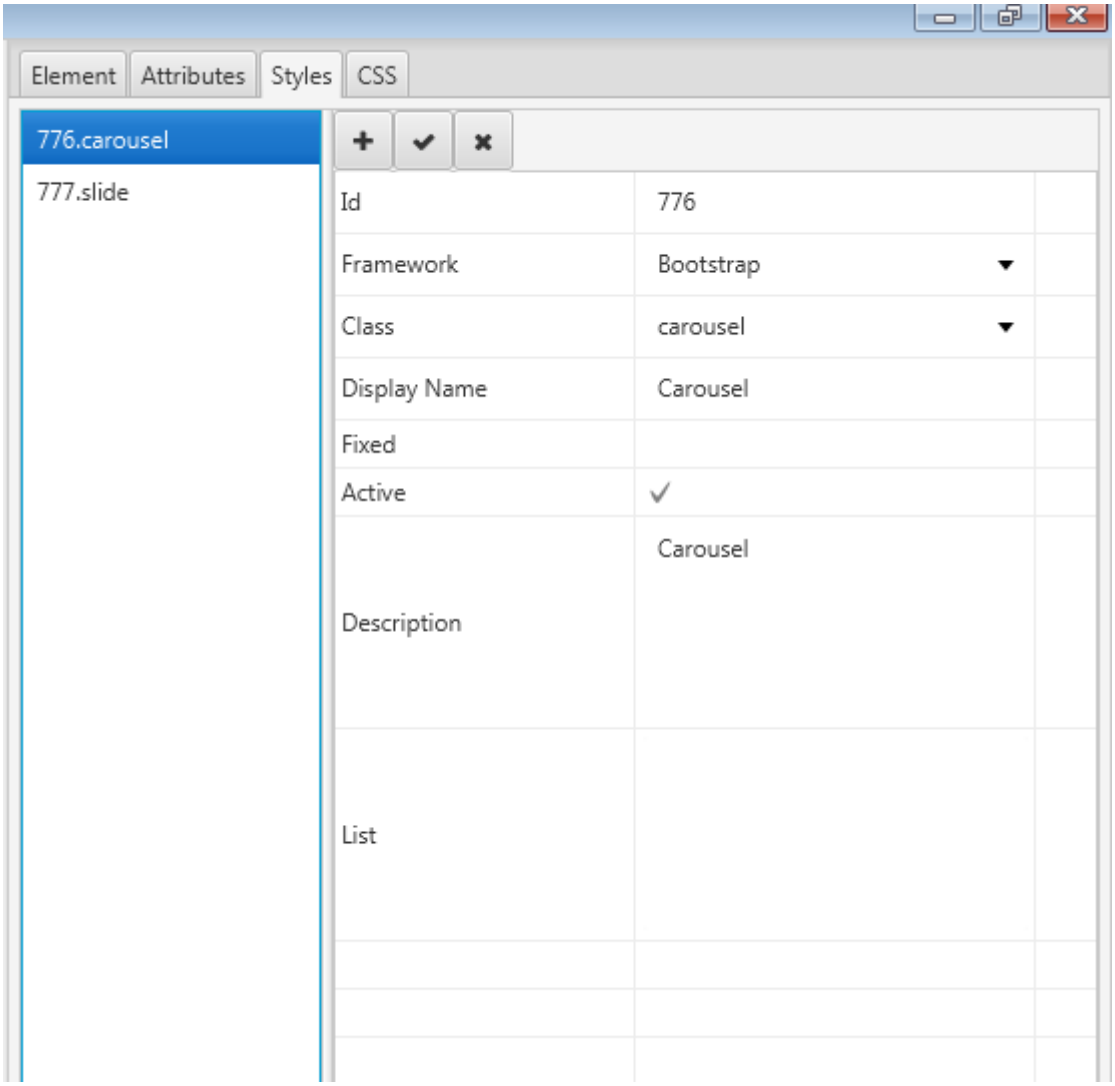
The element styles as seen in the Attributes screen can also be defined here. For this element, a **carousel** attribute has been added with the various properties.

To add a new style, click the add button

1.  Add style (these are style classes defined in the css files)

2.  Save style (after capturing the properties)

3.  Delete style



CSS

App > Creator > Elements > CSS Property Bag

The element css is for inline css that can be added at element level or page level. For this element, a **carousel-inner img class** has been defined has been added with the various properties. As this is defined for the page, in the head section of the page the style will be added as.

```
<style>
      .carousel-inner img {
         width: 100%;
         height: 100%
      }

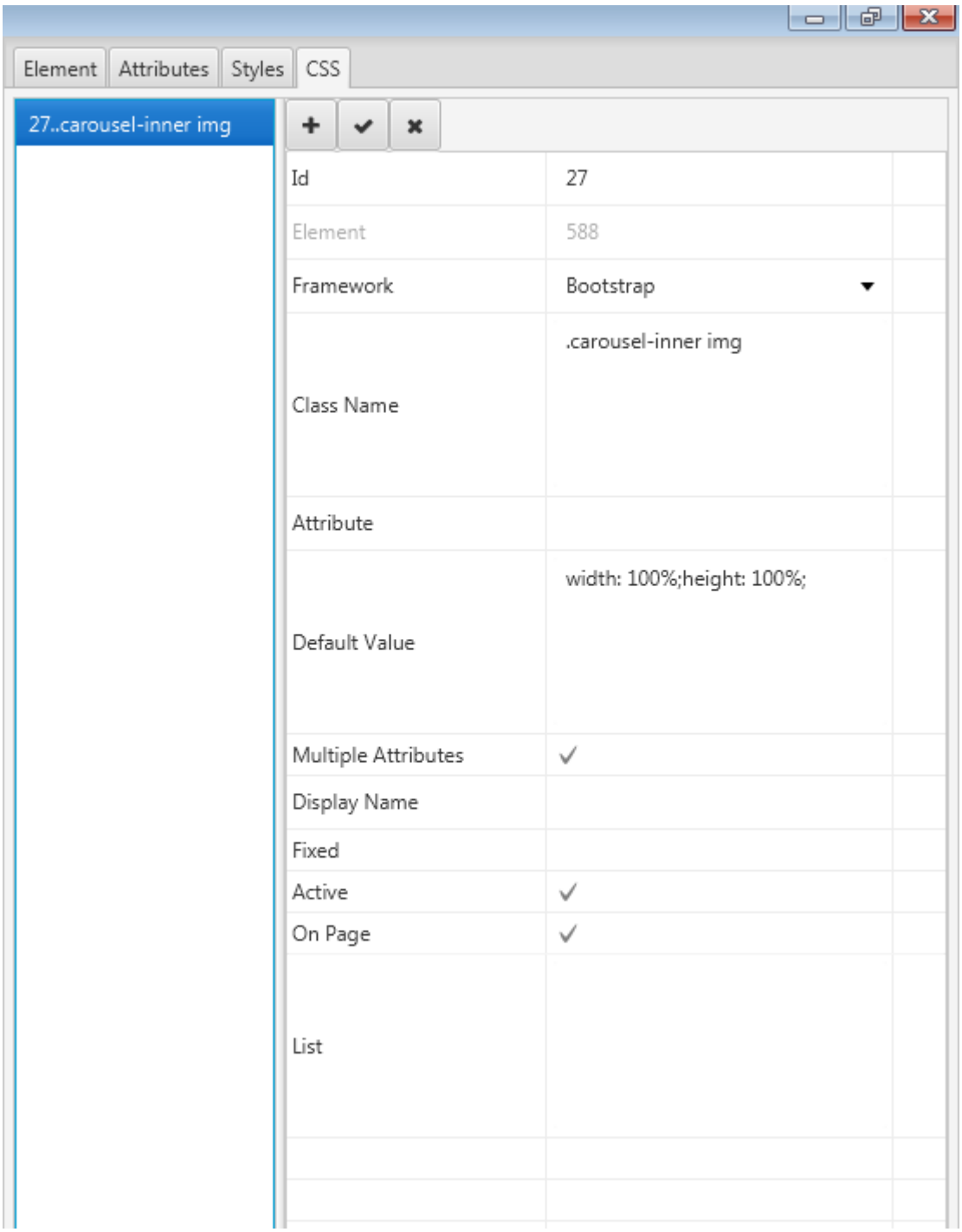
   </style>
```

To add a new css inline property, click the add button

1.  Add css style. This can be added directly to the page (turn On Page on)

2.  Save style (after capturing the properties)

3.  Delete style


Element
Attributes
Styles
CSS
27..carousel-inner img
Id
27
Element
588
Framework
Bootstrap
Class Name
.carousel-inner img
Attribute
Default Value
width: 100%;height: 100%;
Multiple Attributes
Display Name
Fixed
Active
On Page
List